Take-Home Assignment: PHI Pseudonymization Pipeline

CollinearAI · Engineering · Oct 30, 2026 · @Amit Kumar

Overview

Build a pipeline that removes protected health information (PHI) from a mixed set of clinical files by replacing it with consistent pseudonyms, keeps the data clinically and analytically useful, proves the output is clean, and explains how it would run on a petabyte.

You get 12 synthetic files. We score your pipeline by running it on those files **and on held-out variants generated the same way** (different people, values and details, same file families), then inspecting the files it writes. Separately, a design doc shows us how you would take the same system to petabyte scale.

Time box: 5 calendar days from receipt. We expect roughly 12–16 hours of focused work. Please do not spend much more; scope down and tell us what you cut.

Two parts: Part A is a working pipeline. Part B is a design doc for scale.

Follow-up: a 60-minute review call where you run your pipeline live on a file you have not seen, make a small change to it, and walk us through your design and trade-offs.

This assignment is deliberately open-ended. We care less about how much you build than about whether what you build is correct, whether you can prove it is correct, and whether you are honest about where it is not.

The input

You will receive one folder, `input/`, containing 12 files from a synthetic health system. All data is synthetic; no real patient, provider or record is described.

The set mixes clinical and administrative formats: spreadsheets, word-processing documents, PDFs, photographs, structured data, free-text notes and healthcare interchange formats (HL7 v2, FHIR, DICOM).

Treat every file as untrusted. Decide for yourself what each file actually is and whether you can process it safely. File names and extensions are not evidence. Not every file can, or should, be published.

The same people appear in several files, written differently each time. Resolve them to one identity where the evidence supports it, and tell us what you do when it does not.

PHI in scope: every identifier in the HIPAA Safe Harbor set (45 CFR 164.514(b)(2)) of the individual or of their relatives, employers or household members. That includes names, record and account numbers, government and insurance identifiers, all elements of dates except the year, ages over 89, geographic subdivisions smaller than a state, contact details, network identifiers, device and biometric identifiers, full-face photographs and any other unique identifying number, characteristic or code.

Clinical content is not PHI and must be preserved. Diagnosis and procedure codes (ICD-10, CPT, HCPCS, NDC, LOINC, RxNorm), lab values and units, medication names and disease names are not identifiers; removing them destroys the dataset. Over-redaction is a defect, not a safe default.

Care providers are not patients. Provider identifiers (names, NPIs) are a distinct class and must be handled under a policy you state, not merged into the patient pseudonym space.

The 12 files are small on purpose. Your pipeline must still behave correctly on much larger files, and A7 tests that directly.

Part A: the pipeline

Run one command over `input/` and get back clean files, quarantined files, and a report that accounts for everything. Each requirement below is scored.

A1. Detect

Find PHI anywhere it can live in a file and survive into the output. Our test for a leak: **if a person opening the output in its native application, using any feature that application offers, or a script parsing it with a standard library, can recover a real identifier, it leaked.** Text rendered into pixels counts. The visible body is only one of many places.

- Expect structure inside structure, and data that is not where the format's common case puts it.
- Use context, not just patterns. A value with no surrounding text can still be PHI if its column, key, field or neighbours say so.
- Be precise. Words that merely look like names, and numbers that are quantities or codes rather than identifiers, should be left alone.
- Be honest about recall. Tell us which kinds of PHI your approach will miss and why.

A2. Pseudonymise

- Replace each identifier with a realistic surrogate of the same kind and, where practical, the same shape (a phone number becomes a valid-looking phone number; an NPI becomes a valid-looking NPI).
- **Deterministic:** the same real value gets the same surrogate on every run, in every file.
- **Identity-aware:** different written forms of the same person should map to one consistent identity. Explain how you decide two mentions are the same entity, and what you do when you are unsure. Two different people can share a name.
- The surrogate mapping must not be reversible by anyone who only has the output. Secrets come from the environment. If you keep a mapping, say where it lives and how it is protected.

A3. Write the output

- Output files keep their original format and stay valid: they open in the native application and parse with standard libraries.
- Preserve structure: columns, keys, schemas, sheet layout and document formatting should survive as far as is reasonable. Note anything you could not preserve.
- Whatever you detected must actually be replaced in the written file, not only in your in-memory model of it.
- For PHI rendered into pixels (photographs, scanned pages, burned-in image text), a realistic surrogate drawn into the image is not required. Masking is acceptable. State your policy.
- If a file cannot be rewritten both safely and validly, quarantine it with a reason. Do not publish a degraded file and call it clean.

A4. Verify, fail closed

- After writing, independently check the serialised output file for remaining PHI. The verifier must not simply trust the detector that produced the edits.
- A file is published only if it passes verification. Anything that fails, errors, times out, or cannot be processed safely goes to quarantine with a machine-readable reason. Nothing is silently skipped, and nothing is published on an exception.
- Your verifier should not flag your own surrogates as leaks, or quarantine clean files for no reason. Tell us its false-positive behaviour.

A5. Operate safely

- **Idempotent and resumable.** We will kill your pipeline with SIGKILL at random points and re-run it. The result must match a clean run: no double-processing, no corrupt or half-written files, nothing lost.
- **Logs and reports must never contain raw PHI.** Mask or hash anything you need to reference.
- **LLM use is bounded.** If you call a model, the pipeline must respect a configurable cost budget and keep working, or fail closed, when the model is slow, rate-limited, returns garbage, or is unavailable.

A6. Account for everything

Produce a run report that lets a reviewer answer "did we miss anything, and what did it cost?" without reading your code. At minimum, per file: detected type, status (published / quarantined) and reason, findings by PHI type, how many were detected versus applied versus confirmed clean by the verifier, and time and LLM cost spent.

A7. Resource limits

We run the pipeline in a container with **2 vCPUs and 1 GB of RAM**, with network access only to the model endpoint.

`tools/make_large_csv.py` generates an arbitrarily large CSV in the same family as one of the inputs (for example `--size-gb 2`). Your pipeline must process it within the memory limit, cover every row without truncating or sampling, and finish within 30 minutes with the model disabled. Tell us your measured peak memory and throughput.

Healthcare de-identification rules

These rules apply to Part A and are where most of the signal is. Removing direct identifiers is necessary but not sufficient: the output must stay usable for clinical analysis. A solution that blanks every field will fail, and so will one that runs only a regex or only a model pass.

1. **Dates: shift, do not blank.** Shift each patient's dates by a single consistent offset so intervals survive (for example, admission to discharge). The same patient's dates appear in several files and in several notations, and must shift identically everywhere. Do not replace dates with a constant placeholder. If a date cannot be attributed to a patient, state what you do with it.
2. **Ages: generalize the tail.** Any age of 90 or older, however it is expressed, must become `90+`. Ages under 90 may remain.
3. **Geography: truncate.** Zip codes keep only the first three digits; for the restricted low-population prefixes defined under HIPAA, use `000`. Other geographic units smaller than a state are identifiers.

4. **Providers vs patients.** Pseudonymize patients. Map providers (by name and NPI) to a separate, consistent provider identity, or retain them, under a policy you state. Never give a provider a patient surrogate. If one human appears in both roles, tell us what you did.
5. **Relatives and household members are PHI** of the patient and must be handled, even when they never appear as a patient themselves.
6. **Preserve clinical codes and values exactly:** ICD-10, CPT, HCPCS, NDC, LOINC, RxNorm, lab results and units, and medication and disease names.
7. **Interchange formats are handled in place.** PHI in HL7, FHIR and DICOM must be treated without breaking the file for that format's standard tooling.

State every policy choice you make (how you handle providers, an age of exactly 90, an unattributable date, pixel PHI) in `NOTES.md`. We score the decision and its consistency, not whether it matches ours exactly.

Part B: design for a petabyte

Write a design doc (max 6 pages, diagrams welcome) for running this pipeline over 1 PB of heterogeneous files sitting in object storage. Answer each question below; we score the reasoning and the numbers, not the length.

1. **Unit of work and parallelism.** What is the unit a worker picks up (file, shard, byte range, record batch)? How is work distributed and scaled horizontally? Which component saturates first, and why?
2. **Shared state.** Your consistency requirement (same entity, same surrogate, across all files) implies shared state. Where is it, how does it avoid becoming the bottleneck, and what happens to it under concurrent writers? Identity resolution across files is part of this question.
3. **Large single files.** Walk through a 2 GB CSV, a 400,000-cell workbook and a 3,000-page scanned PDF. How do you bound memory, cover every unit without truncating or skipping, and stay within a per-file time and model budget? Your A7 measurements should be consistent with what you claim here.
4. **Cost.** Give a $/TB estimate and a total for 1 PB, with your assumptions written down, including OCR. How do you keep model calls from dominating, and how is a hard budget ceiling enforced across the fleet?
5. **Rate limits.** Model APIs have fleet-wide token and request limits. How do workers share them, and what happens when you add workers beyond that ceiling?
6. **Failure and recovery.** A worker dies on file 4 of 7 million. What happens? Cover checkpointing, retries, duplicate work, dead letters and backpressure.
7. **Proving correctness.** How do you show, with evidence, that no PHI leaked across the whole petabyte and that no file was silently dropped? How do you know the run is finished?
8. **Data locality and storage.** Where does compute run relative to the data? How are outputs written, versioned and kept immutable?
9. **Observability.** Which metrics and alerts would you watch during a multi-week run? Note that file counts are a poor proxy when file sizes vary by 1000×.
10. **The long tail.** Thousands of formats, encodings and corrupt files. What is the plan for anything you do not support?

Where your Part A implementation would not survive at this scale, say so and say what you would change. Knowing the gap is worth more than hiding it.

Deliverables

Submit a private Git repository (or zip) containing the items below. We will run it on a clean Linux machine with Python 3.11+ or Docker.

Item	What we expect
Source code	Any language; Python preferred. Tests for the parts you consider risky.
README.md	One command to run the pipeline end to end, plus setup and environment variables.
output/ from your own run	The exact result of running your pipeline on the 12 files, committed so we can compare it with our rerun.
DESIGN.md (or PDF)	Part B, max 6 pages.
NOTES.md	Max 1 page: known limitations and expected misses, every policy decision, what you cut for time, what you would do next, and your A7 measurements.

Expected output layout:

```
output/
  published/     # clean files, same names and formats as input
  quarantine/    # files not published, each with a <name>.reason.json
  report.json    # per-file and run-level accounting (see A6)
  run.log        # no raw PHI
```

If your pipeline needs secrets to run (a mapping key, an API key), read them from the environment and never commit them.

How we evaluate

We re-run your pipeline on the same 12 files and on held-out variants, under the A7 limits, with crashes injected and with the model degraded or disabled. We then check the output files themselves against a ground-truth list of every planted identifier, every clinical value that must survive, and the per-patient dates. We score both what you removed and what you kept.

Area	Weight	What we look at
PHI removal and verification	25%	Leaks in the serialised output across every surface; an independent verifier that fails closed
Healthcare de-identification correctness	20%	Per-patient-consistent date shifting across files and notations; 90+; geography; provider vs patient; relatives; clinical codes and values intact
Scalability and parallelism design	20%	Part B: unit of work, shared state, large files, failure recovery, proof of completeness
Edge-case and format depth	15%	Hidden surfaces; PDF, image, HL7, FHIR and DICOM depth; type sniffing; precision on decoys; cross-file identity
Robustness	10%	A7 memory and time limits; SIGKILL and re-run; model slow, rate-limited or unavailable

Area	Weight	What we look at
Cost and rate-limit reasoning	5%	A defensible $/TB and 1 PB estimate; budget enforcement; fleet-wide rate limiting
Observability and reconciliation	5%	report.json accounts for every finding, every file and what was preserved

A few things weigh heavily regardless of polish:

- A leak in a file you marked as published counts far more than a file you quarantined with a clear reason.
- A report that says "clean" when the file is not is worse than a report that says "unsure".
- Results on the held-out variants count as much as results on the files you were given.
- Clear, honest limitations in NOTES.md earn credit. Undisclosed gaps we find ourselves do not.

Ground rules and FAQ

Libraries and tools. Use any open-source libraries (NER models, OCR engines, document parsers, etc.). Say what each one is for. AI coding assistants are allowed and we assume you will use them. You must be able to explain and change every line in the review call.

LLMs. Optional. If you use one, we provide the key: read it from `LLM_API_KEY` (and `LLM_BASE_URL` / `LLM_MODEL` if you need them; values are in your invite email). The key has a hard spend cap of **USD 3** for a run on the 12 files, enforced on our side, so design for cost from the start. We will also run with the key unset, with the endpoint rate-limited, slow, and revoked part-way through. In every case the pipeline must produce a safe result: publish or quarantine, never a leak. Make `LLM_ENABLED=0` disable model calls entirely, and report tokens and cost per file.

Do not hardcode. Logic tuned to the specific names, values or layouts in our files will be caught on the variant set and scored as zero for those cases.

Questions. Email amit@collinear.ai within the first 48 hours. If something is ambiguous, make a reasonable call, write it down in NOTES.md and move on; we score the decision, not whether you asked.

Submission. Share the repo link with amit@collinear.ai by the deadline in your invite email. We will confirm receipt within one working day and schedule the review call.

Review call (60 min). 20 minutes: we hand you a file you have not seen and you run your pipeline on it live, then make one small change we ask for. 25 minutes on Part B. 15 minutes on what you would change. Be ready to discuss one file where your pipeline made a judgement call, and to explain any line of your verifier we point at.